

Medical Triage Agent

医疗分诊与问诊智能体

课程名称	企业级应用软件设计与开发
项目名称	Medical Triage Agent：医疗分诊与问诊智能体
方向	方向一：Agentic AI 原生开发
学号	TODO-请填写学号
姓名	TODO-请填写姓名
专业	TODO-请填写计算机技术或软件工程
指导教师	戚欣
提交日期	2026 年 6 月 22 日

CS599 企业级应用软件设计与开发期末大作业

目录

1. 选题背景与设计思想
2. Specs 规格文档
3. 系统架构与设计
4. 关键实现与代码展示
5. 测试与评估
6. 系统升级与扩展
7. 课程总结

提交前请将封面页中的学号、姓名、专业替换为真实信息，并用 PDF 阅读器检查导航目录或书签是否可用。

一、选题背景与设计思想

1.1 问题定义

普通用户在出现头痛、发热、腹痛、咳嗽等常见症状时，往往难以判断是否需要立即就医。搜索引擎答案碎片化，普通聊天机器人又可能忽略红旗症状。本项目构建一个医疗分诊与问诊智能体，在非诊断边界下帮助用户完成初步信息收集、风险提示和就医建议。

1.2 现有方案不足

搜索式问答缺少连续追问；普通 LLM 问答缺少安全门控；纯规则分诊表达僵硬；单次问答无法沉淀问诊记录，不利于复盘和评估。

1.3 项目价值

Medical Triage Agent 将规则、RAG、LLM 和状态化对话结合起来：先保障安全，再追问补全，随后检索医学知识并进行风险分级，最后生成可读建议，体现 Agentic AI 从规格、架构、实现到评估的工程闭环。

1.4 技术路线

项目采用 FastAPI + React 架构，后端通过 Orchestrator 编排多个专职 Agent。医学知识使用 Markdown 文档维护，并通过 Chroma 向量库检索。LLM 使用 DashScope 兼容 OpenAI 接口的 Qwen 模型。SQLite 保存会话、消息和反馈。

二、Specs 规格文档

完整规格见 docs/specs.md，覆盖 Product Spec、Architecture Spec 和 API Spec。

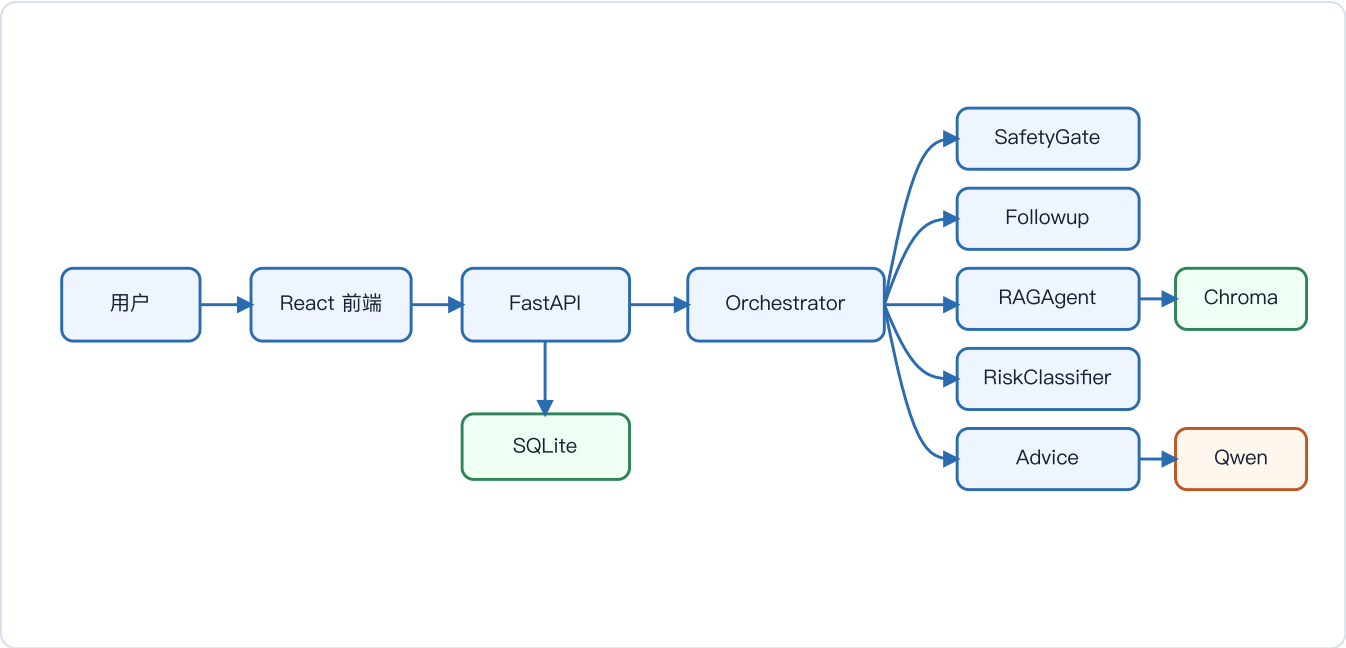
编号	需求	优先级	验收标准
P1	识别红旗症状	Must	胸痛、呼吸困难等输入直接输出急救建议
P2	多轮追问补全信息	Must	信息不足时追问部位、时长、严重程度等
P3	RAG 检索医学知识	Must	从本地医学文档检索相关知识片段
P4	输出风险等级	Must	输出 A/B/C/D 等级、分数和风险因素
P5	生成健康建议	Must	基于风险等级和 RAG 上下文生成非诊断建议
P6	保存对话与反馈	Should	SQLite 持久化会话、消息和评分

Agent 分工

Agent	职责	输出
SafetyGate	紧急安全门	红旗症状、急救建议
FollowupAgent	信息补全	缺失字段、追问问题
RAGAgent	医学知识检索	相关知识片段
RiskClassifier	风险分级	风险等级、分数、因素
AdviceGenerator	建议生成	健康建议
SummaryWriter	摘要生成	结构化摘要

三、系统架构与设计

3.1 核心架构图



3.2 Agent 交互流程

- 1. 用户输入症状。
- 2. SafetyGate 检查红旗症状；命中则立即返回急救建议。
- 3. FollowupAgent 检查症状、部位、持续时间、严重程度和伴随症状是否完整。
- 4. RAGAgent 从医学知识库检索相关片段。
- 5. RiskClassifier 生成 A/B/C/D 风险等级与风险因素。
- 6. AdviceGenerator 基于用户信息、风险等级和 RAG 上下文生成建议。
- 7. API 层保存对话、风险结果和反馈。

3.3 数据流设计

自然语言输入经过 Pydantic 校验后进入会话上下文；Agent 输出结果写入 SQLite；医学知识文档被切分并写入 Chroma；日志记录每次问诊的关键状态。

四、关键实现与代码展示

4.1 Agent 核心循环

```
has_red_flags, detected_flags = self.safety_gate.detect_red_flags(message)
if has_red_flags:
    return {"response": self.safety_gate.get_emergency_response(detected_flags),
            "is_emergency": True, "is_complete": True}

missing_fields = self.followup_agent.identify_missing_fields(conversation_context)
if missing_fields:
    followup_question =
self.followup_agent.generate_followup_question(missing_fields)
    return {"response": followup_question, "needs_followup": True, "is_complete":
False}

rag_context = self.rag_agent.retrieve(message)
risk_report = self.risk_classifier.generate_risk_report(user_info)
advice = self.advice_generator.generate_advice(user_info, rag_context,
risk_report["risk_level"])
```

4.2 工具与模块化调用

项目没有把所有能力写成一个 Prompt，而是将每个 Agent 设计为可函数化调用的模块。SafetyGate、RAGAgent、RiskClassifier 都可以单独测试，也可以作为未来 MCP Tool 或 LangGraph Node 接入。

4.3 配置文件

配置集中在 src/app/utils/config.py，API Key 通过 .env 注入，不硬编码。.gitignore 已排除 .env、日志、数据库、向量库和 node_modules。

4.4 AI IDE 使用

开发过程使用 AI IDE 辅助完成代码审查、规格文档整理、测试用例补齐和交付结构检查。最终代码仍通过本地测试和人工审阅保证一致性。

五、测试与评估

5.1 功能测试

测试目录为 tests/，覆盖 SafetyGate、RiskClassifier 和 FollowupAgent。

场景	输入	预期
急症识别	胸痛、呼吸困难	立即输出 120 和急诊建议
信息不足	我不舒服	追问关键病情信息
普通咨询	轻微咳嗽、有痰	输出低风险和观察建议
高危人群	老年人发热、有基础病	风险等级上调

5.2 行为评估指标

重点观察红旗症状召回率、追问有效性、风险因素可解释性和建议边界声明。系统日志记录咨询 ID、红旗症状、缺失字段、风险等级等关键事件，可用于复盘 Agent 决策链路。

六、系统升级与扩展

1. 将 Orchestrator 升级为 LangGraph 状态机，显式管理节点、边和中断恢复。
2. 将 SafetyGate、RAGAgent、RiskClassifier 封装为 MCP Tool，支持被其他 Agent 复用。
3. 引入更严格的医学安全评测集，对红旗症状召回率和误报率进行量化。
4. 增加 Docker Compose，一键启动 API、前端和向量库。
5. 增加 Demo 视频和云服务器部署 URL，提升最终展示稳定性。

七、课程总结

通过本项目，我从“调用一次大模型生成答案”的思路，转向“设计一个可控、可评估、可扩展的智能体系统”。在医疗场景中，Agent 首先要处理安全边界，再考虑回答质量。SDD 方法帮助我把产品目标、架构、API 和测试对齐，使实现过程更接近工程闭环。后续我希望继续加强 LangGraph、MCP 和可观测评估能力，让系统从课程 Demo 进一步走向可复用的垂直领域 Agent 框架。